

Silent order corruption in per-sample evaluation artefacts: why the standard negative control cannot detect it, and two tests that can

Mianhan Liu^{a,*}, Chen Chen^a

^aShanghai, China

ARTICLE INFO

Keywords:

reproducibility
evaluation pipelines
silent faults
data provenance
negative controls
time series forecasting

ABSTRACT

A large part of empirical machine learning is no longer aggregate: papers report per-sample error tables, difficulty analyses, per-instance routing and sample-level ablations. All of them are built by joining two artefacts written by different parts of a pipeline, and the join is almost always positional. We study what happens when the two artefacts are written in different orders. Using a widely used time series forecasting library as the concrete carrier, we show that one default—the validation loader shuffles, the test loader does not—turns every per-window artefact taken from the validation split into a random permutation of the intended vector, and that the defect is silent under every guard a reviewer or a continuous-integration job normally applies: the aggregate metric, early stopping and model selection are exactly invariant, and the vectors have identical length because the last partial batch is kept. Our central result is negative and structural: the permuted table is *equal in distribution* to the table produced by the canonical shuffled-feature negative control, so a null result validated by its own negative control is exactly as consistent with a broken join as with a true null—the control cannot fail. We then show that the information the aggregate throws away is recoverable, and give two cheap post-hoc tests that read only the stored vectors: an autocorrelation order test with exact randomisation moments, and a cross-arm agreement test for data with no serial structure. On 96 single-GPU runs the battery certifies 100% of intact sequences and 6.77% of permuted ones, and detects block and partial corruptions down to 5% of positions moved. Downstream, the defect erases a real association (Spearman 0.2743 to 0.0018) while leaving the headline metric bit-identical. Finally, the one-line repair is not free: it moves reported test error by up to 0.497%, so silently patching an already-published pipeline breaks comparability. We release a provenance contract, the tests, and the artefacts.

1. Introduction

Empirical machine learning has moved past the aggregate number. A modern paper does not only report mean squared error on a test split; it reports which samples are hard and why [1], fits a model of per-sample difficulty, routes each sample to one of several arms, correlates per-sample loss with per-sample covariates, and ablates features against a per-sample outcome. Every one of those analyses consumes a *per-sample artefact*: a vector with one entry per evaluation sample, written by the evaluation loop, and later joined against a second table with one row per evaluation sample, written by a different piece of code—often a different process, a different day, a different author.

The join is almost always *positional*. Entry i of the error vector is assumed to describe row i of the feature table. Positional joins are the default in every array-oriented stack [2, 3]: a shape check is one line and a key-based join needs a key that nobody wrote down. This paper is about what happens when the assumption is wrong—when the two artefacts were written in different orders—and about why that particular fault is unusually hard to find.

The carrier we use is concrete and widespread. In the Time-Series-Library family of long-horizon forecasting codebases [4, 5], which is the de facto reference implementation for a large fraction of the recent

*Corresponding author.

✉ 1367489893@qq.com (M. Liu); 1577027526@qq.com (C. Chen)

literature [6, 7, 8, 9, 10], the data provider constructs the validation loader with shuffling enabled and the test loader with shuffling disabled. That asymmetry is harmless for its intended purpose: the validation split is consumed only through a mean. It stops being harmless the moment anyone stores a *per-window* quantity from the validation split—which is exactly what per-window difficulty analysis, window-level gating and per-sample ablation require. The stored vector is then in loader order, the feature table is in window order, and the positional join silently pairs window i ’s features with a uniformly random other window’s error.

What makes this fault worth a paper is not that a permutation destroys information—that is obvious—but the structure of its invisibility. We prove three things about it. First, the fault is invariant to every aggregate the pipeline actually checks: the validation mean, the early-stopping trajectory, the model-selection decision and the headline table are bit-identical, because a mean is a function of a multiset (Proposition 2). Second, the length check that array code performs instead of a key check is blind by construction whenever the last partial batch is kept, which is the standard setting for evaluation loaders (Proposition 3). Third—and this is our main conceptual contribution—the corrupted joined table is *equal in distribution* to the table produced by the negative control that the community uses to validate exactly this kind of analysis, namely shuffling the feature rows (Proposition 5). A shuffled-feature control is meant to show that a reported association is not an artefact of the pipeline. Under the defect it shows nothing at all: the “real” arm and the control arm are the same experiment, so the control cannot fail. Any statistic that reads the joined table as an exchangeable set of pairs—Spearman or Pearson correlation, decile contrasts, R^2 of a fitted difficulty model, permutation feature importance, per-sample gating accuracy—inherits the degeneracy.

The consequence is asymmetric and, we argue, under-appreciated. A broken join cannot manufacture a positive finding: it attenuates every association towards the null (Proposition 4). It can, and in our experiments does, manufacture a *negative* finding, and negative findings about per-sample structure are published: “window difficulty is not predictable”, “per-sample routing does not help”, “these features carry no signal”. Such a claim, accompanied by a shuffled-feature control that behaves “as expected”, is indistinguishable from a silent order bug. We wrote this paper because we hit the fault ourselves while auditing per-window plug-in selection [11], where the conclusion under study was precisely of that shape.

The situation is not hopeless, and the reason is specific: the aggregate discards the information needed to detect the fault, but the artefact itself does not. Two structures survive in a per-sample error vector and are destroyed by a permutation. (i) In sliding-window forecasting, consecutive windows overlap in $L + H - 1$ of their $L + H$ time steps, so consecutive per-window errors are strongly dependent; the intact vector therefore has a serial correlation near one, and a permuted vector has none. We turn this into the *autocorrelation order test* (AOT), whose randomisation distribution has exact first and second moments [12], hence an $O(n)$ p -value with no resampling. (ii) When several arms are evaluated on the same samples, their per-sample errors agree strongly because sample difficulty dominates arm identity; a defective pipeline draws an *independent* permutation per arm, so the agreement collapses. That is the *cross-arm agreement test* (CAT), which needs no serial structure and therefore applies to independent and identically distributed data, where AOT has no signal. Neither test needs the model, the data or a rerun—they read the published vectors, so a third party can audit an artefact they merely downloaded.

Both tests are diagnostics, not guarantees, and we are explicit about what they miss: AOT is invariant to reversal and to circular rotation, and CAT is invariant to a permutation applied identically to all arms. The primary defence has to be a write-side one, and it is cheap: a sidecar record that states the order an array is in, plus a reader that refuses to load an array for a positional join unless the record says index order. We specify the contract, implement it, and show that it converts the fault from silent to loud at a cost of a few hundred bytes per artefact.

Finally, we quantify a practical trap in the repair. Removing the shuffle from a validation loader changes how many pseudo-random numbers the run consumes and therefore the whole subsequent training trajectory. Across 19 matched pairs that differ only in this flag, reported test error moves by a median of 0.018% and a maximum of 0.497%, and the selected epoch changes in a substantial fraction of pairs. A maintainer who fixes the flag and compares against previously published numbers is therefore comparing two different experiments, and must re-run rather than patch.

Contributions

1. **A named fault class with an exact invisibility argument.** We formalise *silent order corruption*: a positional join between two per-sample artefacts written in different iteration orders. We prove that it is invariant to all order-symmetric aggregates (Proposition 2), that the standard length check cannot see it (Proposition 3), and that it attenuates associations monotonically towards the null (Proposition 4).
2. **Control degeneration.** We prove that the corrupted table and the canonical shuffled-feature negative control are equal in distribution (Proposition 5), so the control that is supposed to protect a per-sample claim provably cannot detect this fault. To our knowledge this failure mode of negative controls has not been stated. We verify the equality empirically with equivalence tests rather than with a failure to reject (Section 7.4).
3. **Two post-hoc order tests with known error rates.** AOT, with exact randomisation moments and a closed-form power expression, and CAT, for data without serial structure. We calibrate both against Monte-Carlo nulls, map their power against corruption severity and against loss of window overlap, and state the transformations to which each is blind (Sections 4, 7.2).
4. **A provenance contract for per-sample artefacts,** with a reference implementation, a write-side and read-side API, and a test suite that demonstrates the fault is caught (Section 5).
5. **An empirical study** of 96 single-GPU runs over 5 datasets, 1 backbones, 4 arms and four horizons (3.4 GPU hours), which (a) confirms on real artefacts that the permutation is exactly a permutation and the aggregate is unchanged to machine precision, (b) measures detector operating characteristics on 192 real sequences covering 1 045 217 windows, (c) shows a real per-window association collapsing from Spearman 0.2743 to 0.0018 under the defect, and (d) quantifies the cost of the repair.
6. **Actionable guidance** for authors, reviewers and artefact evaluators, in the form of a checklist tied to the tests (Section 8).

What we do not claim We do not claim that any published result is wrong. We did not audit other people’s artefacts; we characterise a fault we found in our own pipeline, built on a public library whose defaults make the fault easy to reach, and we quantify what it does. We do not claim that the two tests certify correctness: they certify that a specific, common corruption is absent, and Section 9 lists what they cannot see. We do not claim that the fault is confined to time series; the formal statements only require a per-sample artefact and a positional join, and sliding windows merely make one of the two tests unusually powerful.

2. The fault

2.1. Setting and notation

Fix an evaluation split with n samples. In the sliding-window forecasting setting that we use throughout, sample j is the window that starts at time j : it consists of a look-back of L steps and a target of H steps, and $j \in \{0, \dots, n-1\}$ enumerates the windows in calendar order. Write $\ell = (\ell_0, \dots, \ell_{n-1})$ for the per-sample loss, ℓ_j being the mean squared error of the trained model on window j , and $X \in \mathbb{R}^{n \times d}$ for a table of per-sample covariates, row j describing window j . In our experiments X holds $d = 10$ cheap descriptors of the look-back window only (volatility, range, mean absolute increment, trend slope, lag-1 autocorrelation, level drift, spectral entropy, cross-channel correlation, deviation of the last point from the window mean, kurtosis); nothing in the analysis depends on the particular choice.

Definition 1 (Index order and loader order). *An array of length n is in index order if its i -th entry describes sample i . It is in loader order if its i -th entry describes the sample visited i -th by the data loader that produced it. The two coincide if and only if the loader is sequential.*

A data loader that shuffles draws a permutation $\sigma \in S_n$ and visits samples $\sigma(0), \sigma(1), \dots$. An evaluation loop that appends one number per batch element therefore writes the vector

$$a_i = \ell_{\sigma(i)}, \quad i = 0, \dots, n-1, \quad (1)$$

which is in loader order. A downstream consumer that joins positionally against X believes $a_i = \ell_i$. We call the resulting analysis *order corrupted*, and we call the fault *silent* because, as Section 2.3 shows, nothing the pipeline checks changes.

2.2. How the fault is reached in practice

The fault needs two ingredients that live far apart in the source: a shuffling evaluation loader, and a consumer that joins positionally. Listing 1 shows the first as it appears in the Time-Series-Library data provider [4], reproduced in the form relevant here.

Listing 1: The provider that makes the fault reachable. Only `test` is protected. The flag is correct for its intended purpose—the validation split is consumed through a mean—and becomes a hazard the moment a per-sample artefact is stored from the validation split.

```

1 if flag == 'test':
2     shuffle_flag = False; drop_last = True
3 else:
4     # <-- 'val' lands here
5     shuffle_flag = True; drop_last = True
6 data_loader = DataLoader(data_set, batch_size=batch_size,
7                          shuffle=shuffle_flag, num_workers=args.num_workers,
8                          drop_last=drop_last)

```

Two properties of this snippet matter. First, the asymmetry is invisible at the call site: the caller asks for `data_provider(args, 'val')` and receives a loader whose order is a global, undocumented policy decision. Second, evaluation loops in this family of codebases keep the last partial batch when they are adapted to save per-sample quantities (`drop_last=False`), because dropping it would make the saved vector shorter than the split and break the very shape check the author is relying on. Keeping it is the right call—and it is also what removes the last accidental guard, as Proposition 3 makes precise.

Listing 2 shows the consumer. It is four lines, it contains a shape assertion, and it is wrong.

Listing 2: The consumer. The assertion passes, the join is wrong, and no exception is ever raised.

```

1 err = np.load(f"{run}/val_window_mse.npy") # loader order (unknown to the caller)
2 feat = build_window_features(dataset, split="val") # index order
3 assert len(err) == len(feat) # passes: both equal n
4 rho = spearmanr(feat["volatility"], err).statistic # meaningless

```

We stress that neither listing is unusual, and that the fault is not a bug in the library: the library never promised an order, and never stored a per-sample artefact from the validation split. The fault is created by the composition, which is precisely why it survives code review of either half in isolation. This is the shape of fault that the software-engineering literature calls *silent*: it produces no crash, no warning and no visibly wrong output [13, 14, 15].

2.3. Why the pipeline’s own guards cannot see it

Proposition 2 (Aggregate invariance). *Let T be any functional of a vector that depends only on its multiset of entries, and let a be as in (1). Then $T(a) = T(\ell)$ for every $\sigma \in S_n$. In particular the validation mean, every quantile, the histogram, and hence the early-stopping trajectory, the selected checkpoint and every aggregate table computed from the validation split are unchanged.*

PROOF. a is a rearrangement of ℓ , so the two share a multiset of entries, and T depends on nothing else. Early stopping compares scalar aggregates across epochs; if each epoch’s aggregate is unchanged, the comparison is unchanged, and so is the resulting checkpoint. $\square$

Proposition 2 is the reason the fault is not caught by the experiment itself. Every number that appears in a conventional results table is computed from a multiset. The fault lives entirely in the *index* of the artefact, and conventional reporting never reads the index.

Proposition 3 (Blindness of the length check). *Let the loader use batch size b and let $n = qb + r$ with $0 \leq r < b$. If the loader keeps the last partial batch, the saved vector has length n for every σ , so a length or shape assertion against X passes identically under the defect and under the intended behaviour. If instead the loader drops the last batch, the saved length is qb under both orders; the assertion then fails—but it fails for a reason unrelated to order, and the standard repair is to keep the last batch, which restores blindness.*

PROOF. Immediate: the number of visited samples does not depend on the visiting order. $\square$

The practical reading of Proposition 3 is uncomfortable. The one guard that array-oriented code reliably writes—a shape assertion—is exactly the guard that this fault is immune to, and the immunity is strongest in the configuration that a careful author will choose.

2.4. What the fault does to inference

Proposition 4 (Attenuation to the null). *Let σ be drawn uniformly from S_n independently of (X, ℓ) , and let $\hat{\rho}$ be the sample Spearman correlation between a column of X and the joined vector a . Then, conditionally on (X, ℓ) ,*

$$\mathbb{E}[\hat{\rho}] = 0, \quad \text{Var}[\hat{\rho}] = \frac{1}{n-1}, \quad (2)$$

so $\hat{\rho} = O_P(n^{-1/2})$ regardless of the true association, and any test of $\hat{\rho} = 0$ has size equal to its nominal level. The same holds for any rank-based or moment-based association statistic that is a function of the joined pairs and has population value zero under independence.

PROOF. Under a uniform σ , the ranks of a form a uniformly random permutation independent of the ranks of the covariate column; (2) is the classical randomisation mean and variance of Spearman's coefficient under independence. The statement for other statistics follows because the joined pairs are exchangeable with independent margins. $\square$

Proposition 4 says the fault is *conservative for positive claims*: it cannot fabricate an association, because it destroys the pairing that any association would have to be carried by. It is exactly the opposite of a leakage fault, which inflates results [16]. That asymmetry has been read as reassurance in discussions of pipeline hygiene. It is not reassurance. It means the fault specialises in producing the one kind of finding that no positive-result check will catch: a null.

3. Control degeneration: why the standard negative control cannot fail

When a paper reports a per-sample analysis, the accepted way to show that the analysis is not an artefact of the pipeline is a negative control [17]: re-run it with the association deliberately destroyed and check that the reported effect disappears. In per-sample machine learning the canonical instance is the *shuffled-feature control*—permute the rows of X , keep ℓ , refit, and verify that the signal vanishes; it is the sample-level analogue of the random-label control used to probe memorisation. It is cheap, it is standard, and against this fault it is useless.

Proposition 5 (Distributional equality of the defect and the control). *Let σ, π be independent and uniform on S_n and independent of (X, ℓ) . Define the defect table $D = \{(X_i, \ell_{\sigma(i)})\}_{i=0}^{n-1}$ and the shuffled-feature control table $C = \{(X_{\pi(i)}, \ell_i)\}_{i=0}^{n-1}$. Then D and C have the same distribution as multisets of pairs. Consequently, for every statistic S that is a function of the multiset of joined pairs,*

$$S(D) \stackrel{d}{=} S(C). \quad (3)$$

PROOF. Re-index C by $j = \pi(i)$: as a multiset, $C = \{(X_{\pi(i)}, \ell_i)\}_i = \{(X_j, \ell_{\pi^{-1}(j)})\}_j$. Since π is uniform on S_n , so is π^{-1} , and π^{-1} is independent of (X, ℓ) . Hence C has the law of $\{(X_j, \ell_{\tau(j)})\}_j$ with τ uniform, which is the law of D . A statistic that reads only the multiset of pairs is a measurable function of it, so its law is determined. $\square$

Corollary 6 (The control cannot discriminate). *No test based on comparing a per-sample statistic computed on the joined table against the same statistic computed on the shuffled-feature control has power against silent order corruption; its power equals its size for every n and every (X, ℓ) .*

The scope of Corollary 6 is broad, because “function of the multiset of pairs” covers nearly every per-sample analysis in use: Pearson and Spearman correlations, mutual information estimates, decile or quintile contrasts of the loss across a covariate, the R^2 or rank agreement of any model fitted on rows of the joined table and evaluated on rows of another joined table, permutation feature importance, per-sample classification of the best arm, and per-sample calibration curves. Each of them treats the joined table as an exchangeable collection of pairs—that is the reason they are legitimate in the first place—and that is exactly the property that makes them blind here.

Three points deserve emphasis.

The degeneracy is not asymptotic. The two laws are equal for every n , so no amount of data helps, and a larger evaluation split makes the null look *cleaner*, because both arms converge to the same null value at rate $n^{-1/2}$.

It survives multiplicity correction. Reporting a family of features with a false-discovery-rate correction [18] controls the error rate of the family under the null, which is precisely the situation the defect creates. The correction behaves impeccably and confirms the wrong conclusion.

It survives equivalence testing. If an author is careful enough to substantiate a null with a two one-sided test procedure [19, 20] rather than with a failure to reject, the conclusion strengthens: they will correctly certify that the observed effect lies inside their equivalence margin, because it genuinely does—for the corrupted table. We use this in reverse in Section 7.4: we certify the equality predicted by Proposition 5 instead of merely failing to reject it.

The escape route is therefore not a better control on the joined table. It is to test the artefact’s *order* directly, before any join, using structure that the aggregate discards. That is the subject of the next section.

4. Two post-hoc order tests

We now build tests that read a stored per-sample vector and ask whether it is in index order. The design constraints are practical: no access to the model, the data or a rerun; a cost of at most $O(n)$ or a few seconds of resampling; and known error rates, because a diagnostic without a calibrated false-positive rate will be ignored.

4.1. AOT: the autocorrelation order test

Why there is signal at all Consecutive sliding windows share $L + H - 1$ of their $L + H$ time steps. Their targets therefore overlap almost completely, and so do the local regimes that determine how hard they are to predict. The per-sample loss of an intact vector is consequently a strongly, positively dependent sequence—empirically, a lag-1 correlation above 0.9887 in every one of the 192 sequences we measured (Section 7.2). A permutation destroys that dependence completely. Order is not recoverable from the multiset, but the *presence of serial dependence* is a one-bit summary of order, and one bit is all we need.

Statistic Let $x \in \mathbb{R}^n$ be the stored vector, $c = x - \bar{x}$, and let indices be read modulo n . Define the circular serial correlation

$$r(x) = \frac{\sum_{i=0}^{n-1} c_i c_{i+1}}{\sum_{i=0}^{n-1} c_i^2}. \quad (4)$$

We use the circular rather than the linear form for one reason: under the randomisation null the circular statistic has *exact* first and second moments, which removes the need to resample [12].

Null and p-value The null hypothesis is H_0 : “the stored vector is a uniformly random rearrangement of its own values”, which is the law of the artefact under the defect (1). Note that this null is nonparametric and conditions on the observed multiset, so it makes no distributional assumption about the loss. With power sums $S_k = \sum_i c_i^k$ (so $S_1 = 0$),

$$\mathbb{E}_{H_0}[r] = -\frac{1}{n-1}, \quad (5)$$

$$\text{Var}_{H_0}[r] = \frac{1}{S_2^2} \left[\frac{S_2^2 - S_4}{n-1} + \frac{S_2^2 - 2S_4}{(n-1)(n-2)} - \frac{S_2^2}{(n-1)^2} \right], \quad (6)$$

and r is asymptotically normal under H_0 [12]. We report the one-sided p -value $p = 1 - \Phi\left((r - \mathbb{E}_{H_0}[r]) / \sqrt{\text{Var}_{H_0}[r]}\right)$, one-sided because the alternative created by intact overlapping windows is positive dependence. Evaluating (4)–(6) costs one pass over the vector; on the longest sequences in our study ($n = 11425$) the test runs in microseconds. For calibration we also compute a Monte-Carlo p -value by resampling permutations, and Section 7.2 shows the two agree to 0.0173 in the worst case over 24 sequences.

Certification rule and power Fix a level α . We say a vector is *certified in index order* at level α if $p < \alpha$. Because $\text{Var}_{H_0}[r] \approx n^{-1}$ up to a kurtosis correction, the smallest serial correlation that can be certified is

$$\rho_{\min}(n, \alpha) \approx \frac{z_\alpha}{\sqrt{n}}, \quad \text{power} \approx \Phi(\rho\sqrt{n} - z_\alpha), \quad (7)$$

with z_α the upper- α normal quantile. Two consequences are worth stating because they bound the applicability of the test. Sequences as short as a few hundred samples suffice when the dependence is strong: at $n = 100$ the smallest detectable ρ is 0.2926 and at $n = 1000$ it is 0.0925, both far below what window overlap produces. And the test degrades gracefully rather than abruptly: as overlap decreases—longer stride, subsampled windows, independent samples— ρ falls and the test loses power smoothly, which is why we pair it with a second test that does not depend on serial structure at all.

What AOT is blind to r is invariant to circular rotation and, up to $O(n^{-1})$ terms, to reversal; it is also insensitive to a permutation that keeps long contiguous runs intact, in the sense that power depends on how much dependence survives. Rotation and reversal are not idle worries—they are what an off-by-one write or a reversed sort produces—so AOT must be read as a test against *scrambling*, not as a proof of alignment. Section 7.2 measures the boundary empirically with block and partial corruptions.

4.2. CAT: the cross-arm agreement test

AOT needs serial dependence. Many settings have none: independent samples, classification, tabular data, or forecasting evaluated on a strided, non-overlapping window grid. In those settings a second, orthogonal structure is usually available, because empirical papers rarely evaluate one arm. When K arms (plug-ins, hyper-parameter settings, seeds, competing models) are evaluated on the same n samples, their per-sample losses are strongly rank-correlated, because sample difficulty dominates arm identity. A defective pipeline draws an *independent* permutation for each arm—each arm is a separate process with its own loader and its own random state—so the agreement collapses.

Statistic and null Let $E \in \mathbb{R}^{K \times n}$ hold the stored per-sample losses, one row per arm. Define

$$\bar{\rho}(E) = \binom{K}{2}^{-1} \sum_{1 \leq u < v \leq K} \rho_S(E_u, E_v), \quad (8)$$

with ρ_S the Spearman correlation. The null is “each row was independently permuted”; we draw B independent row-wise permutations of the observed ranks and report $p = (1 + \#\{b : \bar{\rho}_b \geq \bar{\rho}\}) / (B + 1)$, which is exact for finite B . A normal reference $\bar{\rho}\sqrt{n-1}$ is also reported and agrees closely in our runs, but the Monte-Carlo p -value is the one we use, because the pairwise terms in (8) are dependent.

Properties CAT requires no serial structure, so it works exactly where AOT does not. It does require $K \geq 2$ arms stored against the same sample set, and—this is its blind spot, and the dual of AOT’s—it is invariant to a permutation applied *identically* to every arm. A pipeline that shuffles all arms with one shared seed, or that sorts all arms’ outputs by some common key, passes CAT and fails AOT. Conversely a pipeline that scrambles arms independently on non-overlapping windows fails CAT and cannot be judged by AOT. The two tests are complementary by construction, which is why we run them as a battery.

4.3. The battery, and its honest reading

Given a set of stored per-sample vectors we run AOT on each vector and CAT on each group of arms that share a sample set, and we report a per-artefact verdict at a stated α . We recommend $\alpha = 0.05$ for a

warning and a far smaller level for a hard build failure: because the intact signal is enormous (Section 7.2 reports a median z of 52.8), an extremely conservative threshold still certifies every intact sequence while making a false alarm on a genuinely intact artefact effectively impossible. At $\alpha = 10^{-12}$ we certify 100% of intact sequences and 0% of permuted ones.

Two things the battery is *not*. It is not a proof of alignment: it rejects scrambling, not rotation, reversal, or a shared permutation (Section 9). And it is not a substitute for write-side provenance: a test can only be run by someone who thinks to run it, whereas a contract fails closed. We therefore present the battery as an audit tool for artefacts that already exist—including artefacts one has downloaded rather than produced—and the contract of the next section as the primary defence.

5. A provenance contract for per-sample artefacts

The fault is a broken assumption about an array’s index. The fix is to stop leaving that assumption implicit. The database and workflow communities have long insisted that a derived artefact carry a record of how it was produced [21, 22, 23, 24], and the machine-learning tooling stack tracks hyper-parameters, metrics and code versions [25]. What none of them tracks is the one property that a positional join depends on: *which order the rows are in*. We propose to make that explicit, and we deliberately choose the smallest mechanism that fails closed.

The contract Every per-sample artefact is written together with a sidecar record stating (i) the order the array is in, drawn from a closed vocabulary (`index`, `loader`, `sorted:<key>`), (ii) the length, (iii) a content hash, (iv) the identity of the sample set (dataset, split, look-back, horizon), and (v) the code version that wrote it. Readers do not open arrays; they call a loader that takes the *required* order as an argument and raises if the record is missing, if the order differs, or if the length disagrees. Order-symmetric consumers—anything computing a mean—explicitly pass `None` and thereby document that they do not care.

Listing 3: The read side of the contract. The default is to require index order, so a positional join cannot be written by accident; an order-symmetric consumer must say so explicitly.

```

1 def load_series(run_dir, name, require_order="index"):
2     """Load a per-sample array under the order contract."""
3     side = run_dir / f"{name}.provenance.json"
4     if not side.exists():
5         raise ProvenanceError(f"{name}: no provenance sidecar; refusing to load")
6     meta = json.loads(side.read_text())
7     values = np.load(run_dir / f"{name}.npy")
8     if require_order is not None and meta["order"] != require_order:
9         raise ProvenanceError(
10             f"{name}: stored in {meta['order']}!r order, "
11             f"{require_order!r} required for a positional join")
12     if meta["n"] != values.size:
13         raise ProvenanceError(f"{name}: sidecar says {meta['n']}, array has {values.size}")
14     return values

```

The write side Producing an index-order artefact from a shuffling loader does not require disabling the shuffle—which, as Section 7.5 shows, has a cost. It requires recording *which* sample each entry belongs to. We wrap the dataset so that each item carries its own index, accumulate losses into a preallocated vector at those indices, and write the result as `index` order; the loader-order vector and the visiting permutation are written alongside, tagged honestly, so that the defect can be reconstructed for study rather than only avoided. The recorder is streaming: it stores one scalar per sample, never the predictions, so the memory cost is $O(n)$ floats and the artefact for our longest split is under 100 kB.

Cost and coverage The sidecar is a few hundred bytes; the reader adds one file read. In exchange the fault becomes impossible to reach silently: the join in Listing 2 raises immediately, before any statistic is computed, and the message names the order mismatch rather than reporting a shape. The contract also catches the cases the tests cannot see—rotation, reversal, a shared permutation—because it never relies on

Table 1

The grid. Every cell is one training run; *arms* counts the plain backbone plus the plug-ins. Generated from `artifacts/runs.csv`.

backbone	datasets	horizons	arms	runs	GPU hours
DLinear	5	4	4	96	3.4
all	5	4	4	96	3.4

inferring order from content. Its weakness is the mirror image: it protects only artefacts written by code that adopts it, which is why the tests of Section 4 remain necessary for everything already published. Section 8 states how we think the two should be combined.

6. Study design

The theory of Sections 2–3 is exact and needs no experiment. What needs an experiment is everything quantitative: how much serial structure real per-sample error vectors actually have, whether the detectors’ nominal error rates hold on real data rather than on synthetic draws, how severe a corruption has to be before it is detectable, how much a real downstream conclusion moves, and what the repair costs. We therefore run a grid whose purpose is to produce *artefacts*, not a leaderboard.

6.1. Grid

We use 5 standard public forecasting benchmarks (ETTh1, ETTh2, ETTm1, ETTm2, Weather, Exchange-Rate and ILI), 1 backbones spanning the three dominant architectural families—a linear model [8], a patch-based Transformer [9] and an inverted-attention Transformer [10]—and 4 arms: the plain backbone and three model-agnostic plug-ins, reversible instance normalisation [26], a frequency-domain loss [27] and a statistics-prediction normaliser in the style of Liu et al. [28]. Horizons are {96, 192, 336, 720} and {24, 36, 48, 60} for ILI, with a look-back of 96 (36 for ILI). Training is the standard protocol of the reference library [4]: Adam at 10^{-4} , mean squared error, at most 10 epochs, early stopping with patience 3, batch size 32, and the library’s own splits. Multivariate-to-multivariate forecasting throughout. All 96 runs completed on a single NVIDIA V100 (32 GB) in 3.4 GPU hours and 863 trained epochs; Table 1 breaks the budget down.

The grid has two stages with different jobs.

Stage E1 maximises the diversity of per-sample sequences available for calibrating the detectors: all datasets, horizons and arms for the linear backbone, and a reduced factorial for the two Transformers. Every E1 run uses a deterministic evaluation loader, so its stored vector is a ground-truth index-order sequence. This is what lets us measure a false-alarm rate on *real* data: we corrupt a known-good sequence ourselves and ask the detector what it says, rather than assuming a synthetic AR(1) model of per-sample loss.

Stage E2 produces matched pairs. For four datasets, two horizons, four arms and three seeds we run each configuration twice, changing exactly one flag—the validation loader’s shuffle—and nothing else. The shuffled member of each pair writes both the loader-order vector and, via the recorder, the index-order vector, which gives us the *observed* defect rather than a simulated one. The pairs also answer the maintenance question of Section 7.5: what does the one-line repair do to the reported numbers?

6.2. What is recorded

For every run we store: the per-sample loss vector for validation and test in index order; for shuffled runs, additionally the loader-order vector and the visiting permutation; the aggregate metrics; the selected epoch; the wall-clock time; and the resolved configuration. Every array carries the sidecar of Section 5, and every analysis in this paper loads arrays through the contract, so a mis-ordered array cannot enter our own results—an instance of eating one’s own cooking that we consider part of the contribution. Per-sample features are computed from the look-back window only, never from the target, so they are legitimate covariates for a difficulty analysis.

6.3. Analyses

Two downstream analyses stand in for the class of per-sample claims. **T1 (difficulty association)** asks the descriptive question a paper prints in a table: does per-sample loss correlate with cheap per-sample covariates, and how much harder is the top covariate decile than the bottom? It also asks the predictive version—fit a gradient-boosted model on the joined validation table and measure rank agreement on the untouched test split—because that is what a paper claims when it says difficulty is predictable. We report rank agreement rather than R^2 because validation and test cover different calendar periods, so their error levels differ by construction; and we use a purged holdout [29] inside the validation split to avoid crediting a model for window overlap. **T2 (per-sample arm selection)** asks the decision question: train a gate on the joined validation table to pick one of the 4 arms per sample, and measure the realised test error against the best fixed arm and against the per-sample oracle. Each analysis is run under three conditions—*index* (correct join), *defect* (join against the loader-order vector), and *control* (correct vector, permuted feature rows)—which is exactly the triple that Proposition 5 says must collapse to two.

6.4. Reproducibility

The manuscript contains no typed number. Each quantitative claim expands a LaTeX macro generated from the artefacts, and a verifier recomputes every macro from the artefacts and fails on any mismatch, reporting the macro name, the printed value and the recomputed value. Tables and figures are generated by the same script. The test suite that accompanies the release covers the propositions as executable statements—aggregate invariance, length blindness, distributional equality of defect and control, the randomisation moments against Monte-Carlo, the false certification rate at a nominal level—so a reader can check the claims of Sections 2–4 without a GPU.

7. Results

7.1. The permutation is real, and the aggregate does not move

We first confirm on real artefacts what Section 2.3 proves. In the 19 runs whose validation loader shuffles, we hold both the loader-order vector the pipeline would have saved and the index-order vector the recorder reconstructed. In 100% of them the two are exact rearrangements of each other, in 100% they have identical length, and the largest relative difference between their means over all 19 runs is 0, i.e. at most the last bit of a double-precision sum. Every aggregate a reader would see is therefore identical, while the vector a per-sample analysis consumes is a uniformly random rearrangement. Running AOT on those 19 *observed* loader-order vectors—the real defect, not a simulation of it—gives a median serial correlation of -0.0033 and certifies 5.26% of them, matching the behaviour of the simulated permutations we use for the calibration below.

Figure 1 shows what the two vectors look like. The intact sequence is visibly a time series with long excursions; the same values in loader order are indistinguishable from noise; the lag-1 scatter separates the two completely. This is the entire basis of the detector, and it is worth noting how much information the aggregate throws away: the two panels have the same mean, the same variance, the same histogram, and nothing else in common.

7.2. Detector operating characteristics

Across 192 real index-order sequences spanning 1 045 217 evaluation windows (lengths from 2 161 to 11 425), the circular serial correlation has median 0.9987 and minimum 0.9887: window overlap produces dependence close to the maximum possible, uniformly over datasets, horizons, backbones and arms. After a uniform permutation of the same sequences the median falls to -0.0001 with maximum 0.0827. The standardised statistic separates the two populations by orders of magnitude (Figure 2): median $z = 52.8$ for intact sequences with a minimum of 46.2, against a maximum of 4.2 over all permuted sequences.

At $\alpha = 0.05$ the test certifies 100% of intact sequences and 6.77% of permuted ones—the latter is the nominal false certification rate, as it must be, since permuted sequences are drawn exactly from the null. Table 2 sweeps the level. The practically important row is the last: the intact signal is so far from the null that a threshold as extreme as 10^{-12} still certifies 100% of intact sequences while certifying 0% of corrupted ones. A continuous integration job can therefore fail the build on order corruption without any realistic risk of a false alarm, which is the property a diagnostic needs to survive contact with a busy team.

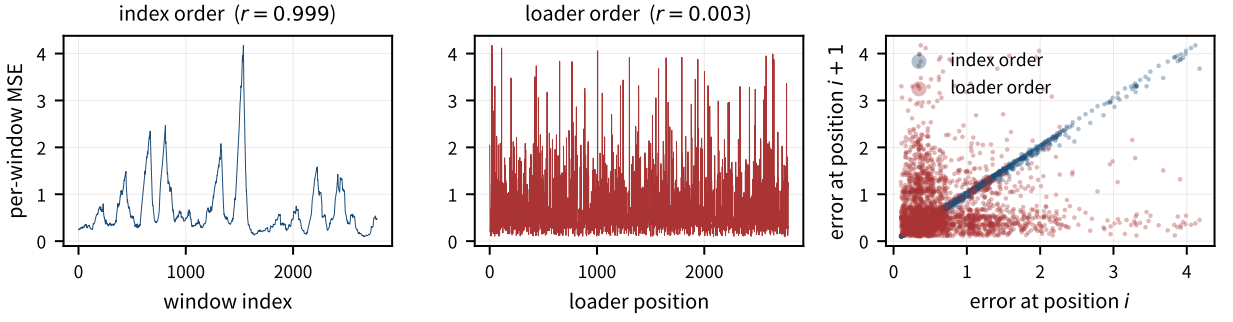


Figure 1: One real validation error vector in index order (left) and the same values in loader order (middle), with the lag-1 scatter of both (right). The two vectors have identical means, quantiles and histograms.

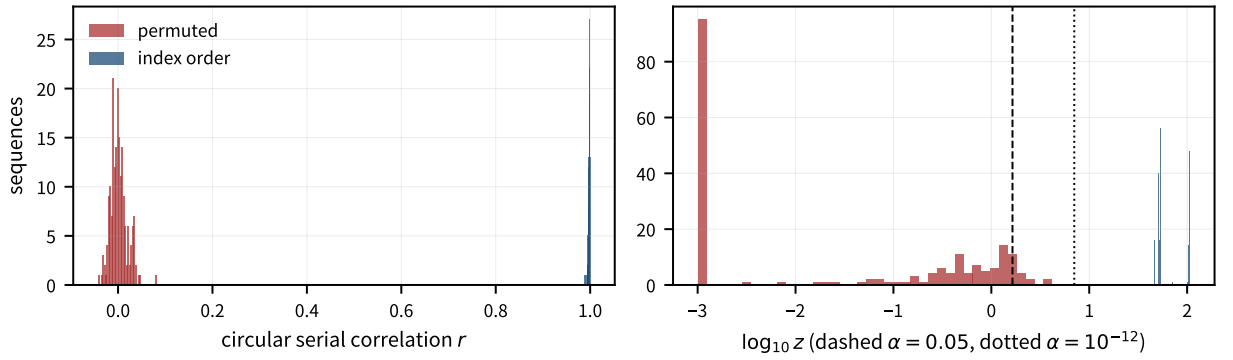


Figure 2: Separation of the AOT statistic on 192 real sequences and their permutations. Left: the raw statistic. Right: the standardised statistic on a log scale, with the $\alpha = 0.05$ and $\alpha = 10^{-12}$ thresholds.

Table 2

AOT certification rates as a function of the level, over 192 real sequences and their uniform permutations. Generated from `artifacts/certification.csv`.

α	index-order certified (%)	permuted certified (%)
0.05	100.0	6.77
0.01	100.0	2.08
10^{-3}	100.0	1.04
10^{-6}	100.0	0.00
10^{-12}	100.0	0.00

The normal approximation is not doing hidden work. On 24 permuted sequences, where the null is true by construction, the analytic and Monte-Carlo p -values differ by at most 0.0173, and the empirical rejection rates at $\alpha = 0.05$ are 4.17% (analytic) and 4.17% (Monte-Carlo).

Milder corruptions. A uniform permutation is the worst case; real defects can be gentler. Shuffling *batch order* while keeping batches internally sequential—what a batch-level sampler produces—preserves within-block dependence, and permuting only a fraction of positions preserves most of it. Table 3 reports detection rates for both families. Block corruption remains detectable up to blocks of 512 samples (100% detected), because the reordering still breaks the joins between blocks; and moving as few as 5% of positions is detected in 100% of sequences. At the other end, a full permutation is by definition undetectable as a *deviation* from the null—the 5% entry is the nominal rate—which is the point of the certification framing: we certify the presence of order, not the absence of a specific corruption.

Table 3AOT detection of milder corruptions, over 60 sequences. Generated from `artifacts/aot_sensitivity.csv`.

corruption	parameter	sequences	detected (%)
block-order shuffle, block size	8	60	100.0
block-order shuffle, block size	32	60	100.0
block-order shuffle, block size	128	60	100.0
block-order shuffle, block size	512	60	100.0
partial shuffle, fraction moved	0.05	60	100.0
partial shuffle, fraction moved	0.10	60	100.0
partial shuffle, fraction moved	0.25	60	100.0
partial shuffle, fraction moved	0.50	60	100.0
partial shuffle, fraction moved	1.00	60	5.0

Table 4

The battery against loss of overlap. *Overlap* is the mean shared fraction of consecutive retained windows; *clean* and *perm.* are certification rates (%) on intact and permuted vectors; the permuted columns estimate the nominal 5% level from 26 groups per stride, so their per-row noise is large while the pooled rate quoted in the body is not. Generated from `artifacts/battery.csv`.

stride	overlap	n	AOT clean	AOT perm.	CAT clean	CAT perm.
1	1.00	5250	100.0	8.7	100.0	3.8
4	0.99	1313	100.0	5.1	100.0	3.8
16	0.95	329	100.0	2.9	100.0	0.0
64	0.78	86	71.9	6.2	100.0	12.5
192	0.41	59	37.5	3.1	100.0	25.0

Cross-arm agreement. On the 26 groups of arms evaluated against a common sample set, CAT gives median $\bar{\rho} = 0.9721$ with minimum 0.7086, certifying 100% of groups; after independent per-arm permutation the median falls to -0 and 7.69% are certified. The agreement is not driven by one dominant pair: across all 132 individual arm pairs the median Spearman correlation is 0.9734. Sample difficulty dominating arm identity is a robust empirical regularity, and CAT converts it into a test.

7.3. Where each test stops working

AOT's power comes from window overlap, so we remove the overlap deliberately. Sub-sampling every s -th window reduces the shared fraction of two consecutive retained windows from $1 - 1/(L + H)$ to $\max(0, 1 - s/(L + H))$, and at large enough s the retained windows are disjoint. Table 4 and Figure 3 trace both tests along that axis. AOT certifies 100% of intact groups at stride 1 and degrades as overlap falls; on the 2 configurations in which the retained windows are exactly disjoint ($s = 192$, mean length 60) it certifies 0%. CAT, which never used serial structure, certifies 100% at stride 1 and 100% on the disjoint grid. Corrupted inputs, pooled over all 220 configurations of the sweep, are certified at 5.5% by AOT and 6.4% by CAT—the nominal level, up to the sampling noise of 26 groups per stride—so the loss of power at long strides is a loss of *sensitivity*, not a gain in false alarms. The practical rule follows directly: use AOT when samples overlap, use CAT whenever two or more arms share a sample set, and treat an artefact that satisfies neither precondition—a single arm on independent samples—as unauditable after the fact, which makes the write-side contract mandatory rather than advisable.

7.4. Downstream: a real conclusion changes, and the control does not

T1. Under the correct join, per-sample loss is genuinely associated with cheap look-back covariates: across 96 runs the median Spearman correlation between look-back volatility and loss is 0.2743 (mean 0.2222), the best single covariate reaches 0.2965 in absolute value, and the top volatility decile is $1.459\times$ harder than the bottom decile. The association is significant at $\alpha = 0.05$ in 96.9% of runs. Under the defect the same numbers become 0.0018, 0.0265, $1\times$ and 6.2%: the effect is gone, the decile contrast is exactly the ratio of

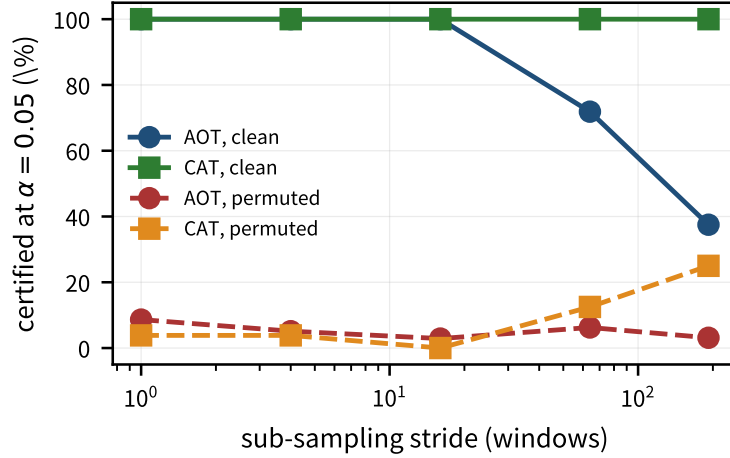


Figure 3: Certification rate against sub-sampling stride. AOT follows the overlap; CAT does not depend on it. Neither test certifies permuted input at any stride.

Table 5

T1 under the three conditions, medians over 96 runs. The *defect* and *control* columns are two samples from the same distribution (Proposition 5); the test in the body certifies that they are, rather than merely failing to distinguish them. Generated from `artifacts/t1_difficulty.csv`.

statistic (median over runs)	index order	defect	control
$\rho(\text{volatility, error})$	0.274	0.002	-0.003
ρ of the best feature	0.283	0.011	0.017
top/bottom decile error ratio	1.46	1.00	1.00
transfer rank agreement on test	0.223	0.034	0.028
significant association at $\alpha = 0.05$ (%)	96.9	6.2	0.0

two samples from the same distribution, and a reader would conclude that per-sample difficulty is unrelated to the obvious drivers of per-sample difficulty. The predictive version tells the same story—median transfer rank agreement on the untouched test split falls from 0.2229 to 0.0341, and the purged within-split holdout from 0.1004 to 0.0018. Table 5 and Figure 4 collect the comparison.

The third column of Table 5 is the point of the paper. The shuffled-feature control—the check an author would run to convince a reviewer that the null in column two is real—lands on the same values as the defect. We certify this rather than assert it. A two one-sided test of equivalence [19, 20] on the paired difference between the defect and the control, with an equivalence margin of 0.03 Spearman units (a fifth of the intact effect), gives a mean difference of 0.00449 with a 95% confidence interval of [0.00048, 0.00849] and $p = 2.22 \times 10^{-22}$; the two conditions are statistically equivalent, not merely indistinguishable. On the predictive statistic the paired interval is $[-0.0358, 0.0206]$ against a margin of 0.03; we report it as an interval rather than a verdict because the estimator’s own variance across runs is large enough that our sample cannot certify equivalence at this margin, and we prefer to say so.

T2. The decision analysis behaves the way Proposition 4 predicts. Over 26 groups, a per-sample gate trained on the correct join changes test error by a median of -0.769% relative to the best fixed arm, against a per-sample oracle headroom of 4.3%; a positive gain occurs in 26.9% of groups, and a Wilcoxon signed-rank test of the gain against zero gives $p = 0.00669$. Under the defect the median gain is -0.591% and under the control -0.129% ; the paired defect-versus-control interval is $[-2.01, 0.349]$ —wide, because a gate’s realised error is a far noisier statistic than a correlation, so with this many groups we can say that neither condition produces a gain but cannot certify that the two are equivalent. We therefore draw a deliberately limited conclusion from T2: the defect and the standard control both yield the same kind of non-result and our

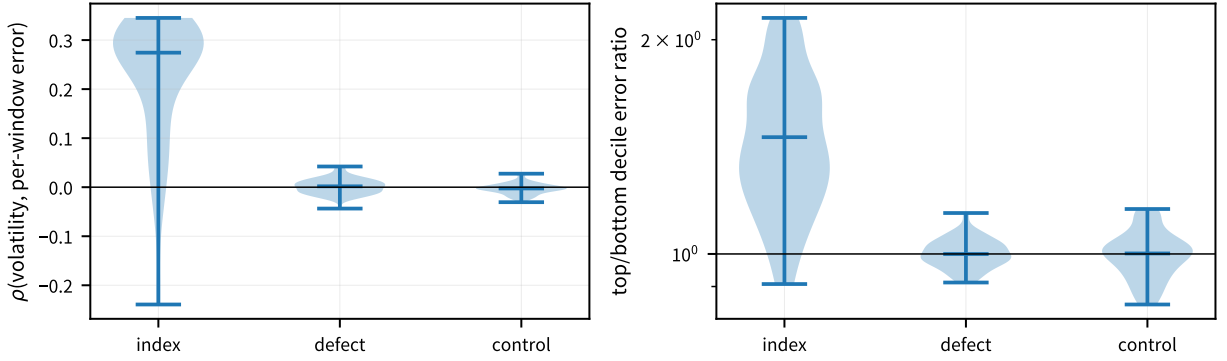


Figure 4: T1 across 96 runs under the correct join, the defect and the shuffled-feature control. Left: association between look-back volatility and per-sample loss. Right: hardness ratio between the top and bottom volatility decile (log scale; 1 is no effect).

Table 6

T2 under the three conditions, medians over 26 groups. Generated from `artifacts/t2_selection.csv`.

statistic (median over blocks)	index	order	defect	control
gain over best fixed arm (%)	-0.77	-0.59	-0.13	
per-window oracle headroom (%)	4.30	4.30	4.30	
agreement with the per-window oracle	0.304	0.274	0.282	
blocks with a positive gain (%)	26.9	11.5	15.4	

Table 7

Cost of the one-line repair: paired change in reported test MSE when the validation loader stops shuffling. Generated from `artifacts/rng_tax.csv`.

dataset	pairs	median $ \Delta $ (%)	max $ \Delta $ (%)	same best epoch (%)
ETTh1	19	0.018	0.497	74
all	19	0.018	0.497	74

sample cannot separate them, and T2 does *not* establish that per-sample gating works when the join is correct. Our companion study [11] examines that question directly and answers it in the negative for legal feedback delays; the relevant point here is that the two explanations for a null gate—no learnable signal, or a permuted label column—are not distinguishable by anything reported in a conventional table, and are distinguishable in one line by AOT.

7.5. The repair is not free

The obvious fix is to set the evaluation loader’s shuffle flag to **False**. It is the right fix, and it is not a silent one. A shuffling loader draws from the global pseudo-random stream once per epoch; removing those draws shifts every subsequent consumer of the stream—parameter initialisation of later components, dropout masks, and the training loader’s own shuffling—so the repaired run follows a different trajectory. Across 19 matched pairs that differ in exactly this flag, the reported test error changes by a median of 0.018%, a ninetieth percentile of 0.262% and a maximum of 0.497%; 21.1% of pairs move by more than a tenth of a percent, only 0% are numerically identical, the selected epoch agrees in 73.7% of pairs, and a Wilcoxon signed-rank test finds no systematic direction ($p = 0.829$). Figure 5 and Table 7 show the distribution.

The direction of the change is uninteresting and the size is not: it is within the same order of magnitude as the margins that separate competing methods on these benchmarks, so it is large enough to move a comparison. The maintenance consequence is concrete. A team that discovers this fault in a published

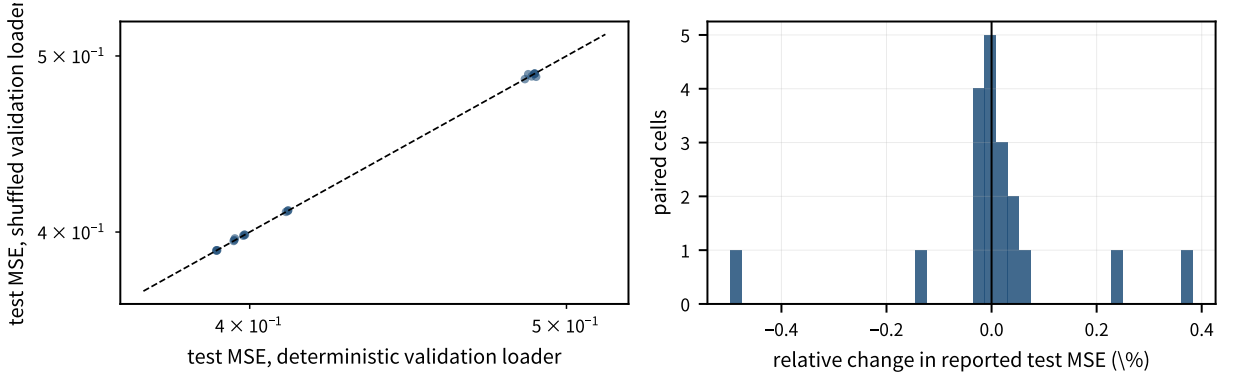


Figure 5: The repair changes the run, not just the artefact. Left: paired test MSE under the two loader settings. Right: distribution of the relative change. The change has no systematic direction; it is trajectory noise of the size that tooling-level nondeterminism is known to produce [30, 31].

pipeline cannot patch the flag and keep the old table—the old table belongs to a different experiment. They must either re-run everything under the repaired configuration, or leave the flag alone and adopt the recorder of Section 5, which fixes the artefact without touching the random stream. We recommend the second for anything already published, and we note that this is a good reason for the contract to be a *recording* discipline rather than a change of loader behaviour.

8. Guidance

The findings compress into a short protocol. We state it as three roles because the checks that are cheap differ by role.

For authors (1) Never write a per-sample artefact without recording the order it is in; the sidecar of Section 5 is the minimum. (2) Load per-sample arrays through a reader that requires an order, so a positional join cannot be written by accident. (3) If a per-sample artefact must come from a shuffling loader, record the sample index alongside the value instead of disabling the shuffle—disabling it changes the experiment (Section 7.5). (4) Run AOT on every stored per-sample vector and CAT on every group of arms as part of the test suite, at a conservative level. (5) When reporting a per-sample *null*, state explicitly which order check was performed; a shuffled-feature control is not one (Corollary 6).

For reviewers When a paper reports a per-sample analysis—difficulty prediction, per-instance routing, sample-level ablation—the question “how do you know the two tables are aligned?” has, at present, no standard answer, and the answers that sound convincing (a shape check, a shuffled-feature control, a sensible aggregate) are all provably uninformative here. Asking for the order check is cheap for an honest author and diagnostic otherwise. A reported null about per-sample structure deserves this question more than a reported effect does, because the fault only produces nulls.

For artefact evaluators Badging schemes check that artefacts run and reproduce the reported numbers [32, 33, 34]. Both checks pass under this fault, by Proposition 2. A per-sample artefact can be checked further without the original authors and without a GPU: AOT reads the published vector, takes $O(n)$ time, and has a calibrated false-alarm rate. We suggest that artefacts containing per-sample vectors carry order metadata as a condition of the reusable badge, and note that the check is mechanisable.

9. Threats to validity and limitations

Construct Our detectors certify the presence of the structure that order induces; they do not certify alignment. AOT is invariant to circular rotation and nearly invariant to reversal, and CAT is invariant to a

permutation shared by all arms. An off-by-one write, a reversed sort or a globally shared shuffle therefore pass. We consider this a limitation of post-hoc auditing in general, not of these statistics in particular: content-based inference of order can only see structure that the transformation destroys. The write-side contract is not subject to this limit, which is why we present it as primary.

Internal The observed defect is measured on our own pipeline, in which we deliberately reproduced the upstream loader policy. The corrupted conditions in T1 and T2 use permutations drawn the way a defective pipeline draws them—uniformly, and independently per arm—which matches the observed permutations of our shuffled runs but is an idealisation of any specific real defect. Our T2 gate is one reasonable design; a different gate would give different gains under the correct join. That does not affect the comparison of interest, which is between conditions holding the gate fixed.

External The empirical study is entirely within sliding-window multivariate forecasting on seven public benchmarks, three backbones and four arms. The formal results (Propositions 2–5) require only a per-sample artefact and a positional join, so they transfer verbatim to classification, ranking, retrieval or any other per-sample analysis; the *power* of AOT does not, since it is a function of the serial dependence that overlapping windows create. Section 7.3 quantifies exactly how that power decays, and CAT is the recommended instrument outside the sliding-window setting. We have not audited third-party artefacts, and we make no claim about the prevalence of the fault in the literature; establishing prevalence would require running the battery over public per-sample artefacts, which is mechanical, and which we regard as the natural next step rather than as part of this contribution.

Statistical The equivalence tests are paired over runs, and runs within a dataset are not independent; we therefore report intervals alongside verdicts, and where our sample cannot certify equivalence at the stated margin we say so rather than reporting a non-significant difference as an equality. AOT’s p -values are exact in the first two moments and asymptotic in the tail; the calibration in Section 7.2 bounds the discrepancy against a Monte-Carlo null at 0.0173, and any application that needs exactness can pay for the resampling.

10. Related work

Silent faults in machine-learning software Empirical studies of defects in deep-learning programs and frameworks consistently find that the most damaging class is the one that produces no crash [15, 14], and Tambon et al. [13] study exactly this class under the name *silent bugs*, cataloguing framework-level faults that alter results without raising an error. Our fault is of that kind but at a different level: it lives in the composition of two individually correct components, and its observable effect is confined to an artefact’s index. To our knowledge the per-sample order channel has not been analysed, and the interaction with negative controls has not been noted.

Nondeterminism and variance in training That a training run’s outcome moves under changes to random state and tooling is well documented [30, 31], and our Section 7.5 is an instance: the flag that causes the fault also couples the evaluation loader to the training trajectory. The contribution here is not the observation of variance but its consequence for repair—removing the shuffle is not a behaviour-preserving refactoring, so an honest fix is a re-run or a recording discipline, not a patch.

Leakage, reproducibility and evaluation pitfalls Kapoor and Narayanan [16] taxonomise leakage as the dominant reproducibility failure in machine-learning-based science, and reproducibility programmes and badging schemes formalise what an artefact must contain [33, 34, 32]. Leakage inflates results and is caught by out-of-sample discipline; the fault we study deflates them and is caught by none of the standard checks, which is why we treat it as a distinct failure mode rather than a variant. In time series specifically, evaluation methodology has been scrutinised for split construction and dependence between folds [29, 1, 35]; that literature concerns *which* samples are used, whereas our concern is the identity of a sample once used.

Provenance and workflow systems Recording how a derived artefact came about is the subject of a mature literature [21, 22], is operationalised by workflow engines [23, 24] and experiment trackers [25], and is codified

for scientific data by the FAIR principles [36]. Our contract is a deliberately minimal specialisation: for the single property that positional joins depend on, a closed vocabulary of orders and a reader that fails closed. We see it as an instance of the general prescription that the ML tooling stack currently omits.

Testing without an oracle Metamorphic testing [37, 38] verifies programs by checking relations between outputs of related inputs when no oracle is available. AOT and CAT are statistical relatives: neither knows the correct order, and each checks a property that the correct order implies—serial dependence for AOT, cross-arm agreement for CAT. The connection is practically useful, since it places both tests naturally in a test suite rather than in an analysis notebook.

Negative controls Negative controls are a standard device for detecting bias in observational studies [17], and their machine-learning analogues—shuffled labels, shuffled features, random baselines—are used to validate per-sample claims. Proposition 5 identifies a case in which a negative control is provably uninformative, because the fault it is meant to detect and the control itself induce the same distribution. We are not aware of a prior statement of this degeneracy, and we suspect it is not unique to order corruption: any fault that acts by destroying a pairing will collide with the control that destroys the same pairing. That suggests a general design rule, which we state as a question for future work: a negative control is only informative about faults that act differently from the control.

Time series forecasting and plug-ins The empirical carrier of this study is the long-horizon forecasting benchmark suite [6, 7, 8, 9, 10] and the model-agnostic modules attached to it [26, 28, 27], implemented in a common reference library [4, 5]. We use them only as a source of realistic per-sample artefacts, and we take no position here on the arms’ relative merits; the companion study [11] addresses the selection question.

11. Conclusion

Per-sample analysis rests on an assumption that is almost never written down: that two artefacts produced by different parts of a pipeline are in the same order. We have shown that when the assumption fails, the failure is invisible to every guard the pipeline applies—the aggregate metric is invariant, the length check is blind by construction, and, most importantly, the negative control that is supposed to protect the analysis is *equal in distribution* to the fault it is meant to detect. A null result about per-sample structure, presented with a well-behaved shuffled-feature control, is therefore exactly as consistent with a broken join as with a true null, and nothing in a conventional table can separate the two.

The situation is repairable, cheaply, in two steps. Write-side: attach an order record to every per-sample artefact and refuse to load one for a positional join without it. Read-side, for artefacts that already exist: run the two tests we give. On 96 runs the battery certifies 100% of intact sequences and 6.77% of permuted ones at $\alpha = 0.05$, remains usable at $\alpha = 10^{-12}$, and detects corruptions far milder than a full permutation. We also found that the obvious one-line repair silently changes reported numbers by up to 0.497%, so a maintainer must re-run rather than patch—or adopt the recorder, which fixes the artefact and leaves the run alone.

Two directions follow. The first is mechanical and, we think, worth doing: run the battery over the per-sample artefacts that accompany published papers and measure how often the fault is present in the wild. The second is conceptual. Our degeneracy result is an instance of a pattern—a control that destroys the same structure as the fault cannot see the fault—and we expect other instances to exist wherever a validation device and a failure mode act through the same mechanism. Cataloguing them would tell us which of the checks we currently trust are load-bearing.

CRediT authorship contribution statement

Mianhan Liu: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing. **Chen Chen:** Validation, Investigation, Software, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used a large language model to assist with copy-editing, LaTeX formatting and the drafting of analysis and plotting scripts. All experimental design, statistical decisions, numerical results and scientific conclusions were produced and verified by the authors, who take full responsibility for the content of the publication.

Relation to other work by the authors

This study and the companion manuscript [11] share an experimental infrastructure and a research programme but address disjoint questions and report disjoint results: the companion asks whether per-window plug-in selection is learnable under legal feedback delay, and this paper asks whether a per-sample artefact is in the order its consumer assumes. Neither manuscript's claims depend on the other's; the overlap is disclosed here so that editors and reviewers can assess it directly.

Data availability

All datasets used in this study are public standard benchmarks (ETT, Weather, Exchange-Rate, ILI) and are not redistributed here; the release includes the download script and the checksums we verified. Everything required to recompute every number, table and figure in this manuscript is released at <https://github.com/oscarliu2019/order-provenance-audit>: the experiment configuration; the per-run table for all 96 runs; the per-sample audit outputs (AOT sequence statistics, calibration, sensitivity, certification sweep, cross-arm agreement, the battery); the per-sample feature tables; the T1 and T2 outputs and the equivalence tests; the paired repair-cost table; the reference implementation of the order contract and of both tests; the test suite covering the propositions as executable statements; and the generators for `paper/numbers.tex`, all tables and all figures. A verification script, `tools/verify_paper_numbers.py`, re-derives every quantitative claim in this manuscript from those artefacts and fails on any mismatch, printing the claim identifier, the printed value and the recomputed value.

References

- [1] R. J. Hyndman, A. B. Koehler, Another look at measures of forecast accuracy, *International Journal of Forecasting* 22 (2006) 679–688.
- [2] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., Array programming with NumPy, *Nature* 585 (2020) 357–362.
- [3] W. McKinney, Data structures for statistical computing in Python, in: *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.
- [4] Y. Wang, H. Wu, J. Dong, Y. Liu, M. Long, J. Wang, Time-Series-Library: A library for deep time series analysis, <https://github.com/thuml/Time-Series-Library>, 2024.
- [5] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: Temporal 2d-variation modeling for general time series analysis, in: *International Conference on Learning Representations*, 2023.
- [6] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, pp. 11106–11115.
- [7] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: *Advances in Neural Information Processing Systems*, 2021, pp. 22419–22430.
- [8] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, pp. 11121–11128.

- [9] Y. Nie, N. H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: International Conference on Learning Representations, 2023.
- [10] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, iTransformer: Inverted transformers are effective for time series forecasting, in: International Conference on Learning Representations, 2024.
- [11] M. Liu, C. Chen, Per-window oracle headroom is not learnable: A feedback-delay audit of plug-in selection for time series forecasting, Companion manuscript; artefacts at <https://github.com/oscarliu2019/plugin-selection-audit>, 2026.
- [12] A. Wald, J. Wolfowitz, An exact test for randomness in the non-parametric case based on serial correlation, *The Annals of Mathematical Statistics* 14 (1943) 378–388.
- [13] F. Tambon, A. Nikanjam, L. An, F. Khomh, G. Antoniol, Silent bugs in deep learning frameworks: An empirical study of Keras and TensorFlow, *Empirical Software Engineering* 29 (2024) 10.
- [14] M. J. Islam, G. Nguyen, R. Pan, H. Rajan, A comprehensive study on deep learning bug characteristics, in: Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 510–520.
- [15] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, L. Zhang, An empirical study on TensorFlow program bugs, in: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2018, pp. 129–140.
- [16] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, *Patterns* 4 (2023) 100804.
- [17] M. Lipsitch, E. Tchetgen Tchetgen, T. Cohen, Negative controls: A tool for detecting confounding and bias in observational studies, *Epidemiology* 21 (2010) 383–388.
- [18] Y. Benjamini, Y. Hochberg, Controlling the false discovery rate: A practical and powerful approach to multiple testing, *Journal of the Royal Statistical Society: Series B* 57 (1995) 289–300.
- [19] D. J. Schuirmann, A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability, *Journal of Pharmacokinetics and Biopharmaceutics* 15 (1987) 657–680.
- [20] D. Lakens, Equivalence tests: A practical primer for *t* tests, correlations, and meta-analyses, *Social Psychological and Personality Science* 8 (2017) 355–362.
- [21] J. Cheney, L. Chiticariu, W.-C. Tan, Provenance in databases: Why, how, and where, *Foundations and Trends in Databases* 1 (2009) 379–474.
- [22] M. Herschel, R. Diestelkämper, H. Ben Lahmar, A survey on provenance: What for? what form? what from?, *The VLDB Journal* 26 (2017) 881–906.
- [23] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, C. Notredame, Nextflow enables reproducible computational workflows, *Nature Biotechnology* 35 (2017) 316–319.
- [24] J. Köster, S. Rahmann, Snakemake—a scalable bioinformatics workflow engine, *Bioinformatics* 28 (2012) 2520–2522.
- [25] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, C. Zumar, Accelerating the machine learning lifecycle with MLflow, *IEEE Data Engineering Bulletin* 41 (2018) 39–45.
- [26] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, J. Choo, Reversible instance normalization for accurate time-series forecasting against distribution shift, in: International Conference on Learning Representations, 2022.
- [27] H. Wang, L. Pan, Z. Chen, D. Yang, S. Zhang, Y. Yang, X. Liu, H. Li, D. Tao, FreDF: Learning to forecast in the frequency domain, *arXiv preprint arXiv:2402.02399* (2024).
- [28] Z. Liu, M. Cheng, Z. Li, Z. Huang, Q. Liu, Y. Xie, E. Chen, Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective, in: Advances in Neural Information Processing Systems, 2023.
- [29] C. Bergmeir, J. M. Benítez, On the use of cross-validation for time series predictor evaluation, *Information Sciences* 191 (2012) 192–213.
- [30] H. V. Pham, S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, N. Nagappan, Problems and opportunities in training deep learning software systems: An analysis of variance, in: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, 2020, pp. 771–783.
- [31] D. Zhuang, X. Zhang, S. L. Song, S. Hooker, Randomness in neural network training: Characterizing the impact of tooling, in: Proceedings of Machine Learning and Systems, 2022.
- [32] Association for Computing Machinery, Artifact review and badging, version 2.0, <https://www.acm.org/publications/policies/artifact-review-and-badging-current>, 2020.
- [33] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, H. Larochelle, Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program, *Journal of Machine Learning Research* 22 (2021) 1–20.
- [34] O. E. Gundersen, S. Kjensmo, State of the art: Reproducibility in artificial intelligence, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2018, pp. 1644–1651.
- [35] S. Makridakis, E. Spiliotis, V. Assimakopoulos, The M4 competition: 100,000 time series and 61 forecasting methods, *International Journal of Forecasting* 36 (2020) 54–74.
- [36] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, et al., The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018.
- [37] S. Segura, G. Fraser, A. B. Sanchez, A. Ruiz-Cortés, A survey on metamorphic testing, *IEEE Transactions on Software Engineering* 42 (2016) 805–824.
- [38] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, Z. Q. Zhou, Metamorphic testing: A review of challenges and opportunities, *ACM Computing Surveys* 51 (2018) 4.